

ROLA Numbered Prompts

Generated from the visible conversation prompts in this workspace. The prompts are numbered in chronological module order.

Prompt 1: Module 13 - Real-Time Translation and Multilingual Learning System

You are an expert Android AI engineer specializing in multilingual AI systems, machine translation, language processing, Kotlin, and AR.

Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation.

Completed modules:

- Android project setup
- MVVM architecture
- Jetpack Compose UI
- Firebase Authentication
- ARCore Camera Scanner
- TensorFlow Lite Object Recognition
- AR Floating Information Overlay
- AI Text-to-Speech Voice Explanation
- Learning History Management
- Room Database
- Firebase Firestore Synchronization
- Search System
- Knowledge Database Exploration
- AI Quiz System
- Gamification
- Complete App Integration
- AI Tutor Chatbot
- 3D Object Visualization System

Now implement Module 13: Real-Time Translation and Multilingual Learning System.

Goal: Enable users to view and listen to educational object information in multiple languages, making the AR learning experience more accessible and engaging.

Requirements:

1. Create translation architecture under presentation/translation, data/translation, and domain/model.
2. Implement translation service for object information, AI tutor response, quiz, and voice explanation translation.
3. Support English, Bengali, Spanish, French, German, Chinese, and Arabic with easy expansion.
4. Create Language model with languageCode, languageName, nativeName, and supportedVoice.
5. Implement TranslationService functions: translateText(), detectLanguage(), getSupportedLanguages(), translateObjectInformation().
6. Create Material 3 language selection UI with current and available languages.
7. Integrate translated object information with AR Information Overlay.
8. Integrate translated text with Text-to-Speech native voice output.
9. Integrate AI Tutor chatbot with language detection and translated responses.
10. Implement offline translation cache and local language packs.
11. Create Room entities LanguageEntity and TranslationCacheEntity.
12. Sync translations to Firestore collection: translations/{languageCode}/translatedContent.
13. Add speech locale availability and fallback handling.
14. Optimize latency, cache usage, network requests, and memory.
15. Secure API communication, service keys, and user input.
16. Test language switching, translation accuracy, voice output, offline cache, chatbot translation, and AR overlay translation.
17. Follow MVVM, Clean Architecture, Repository pattern, Kotlin best practices, and production-level implementation.

Do not modify existing AR scanning, ML recognition, or 3D visualization modules. Only add Real-Time Translation and Multilin

User note: aita ki implement kora ses, ha kora hoile koren akhon.

Prompt 2: Module 14 - AR Glasses and Wearable Device Compatibility

You are an expert Extended Reality (XR) software architect specializing in Android ARCore, wearable computing, AR glasses, and smart glasses.

Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation.

Completed modules include Module 13 Real-Time Translation System.

Now implement Module 14: AR Glasses and Wearable Device Compatibility.

Goal: Prepare ROLA architecture for future integration with AR glasses and wearable XR devices, allowing hands-free immersion.

Requirements:

1. Create wearable architecture under presentation/wearable, data/wearable, and domain/model.
2. Design a hardware abstraction layer for AR glasses, smart glasses, XR headsets, and future wearables without manufacturing dependencies.
3. Create WearableDevice model with deviceId, deviceName, manufacturer, connectionType, batteryLevel, displayCapability, and displayType.
4. Implement Bluetooth/WiFi discovery, connection, and status monitoring states: Connected, Disconnected, Connecting, Error.
5. Create wearable data communication flow: AR glasses camera -> Object Recognition -> Learning Information -> AR Display.
6. Prepare hands-free interaction with voice commands and future hand tracking, touch gestures, and controllers.
7. Adapt UI for small field-of-view transparent displays with minimal text and low distraction.
8. Support mobile floating information cards and AR glasses spatial anchored holographic panels.
9. Implement voice-first learning with voice navigation, spoken explanations, and audio feedback.
10. Add battery optimization, low power mode, background optimization, and adaptive processing.
11. Create device settings for connected device, display mode, audio output, voice sensitivity, and interaction mode.
12. Prepare APIs for eye tracking, hand tracking, spatial mapping, and depth sensors without implementing unavailable hardware.
13. Ensure secure pairing, encrypted communication, and privacy protection.
14. Test device connection simulation, data transfer, AR overlay adaptation, voice interaction, and performance impact.
15. Follow Clean Architecture, MVVM, Repository Pattern, hardware abstraction principles, and Kotlin best practices.

Do not replace existing mobile AR functionality. Only prepare ROLA for future AR glasses and wearable integration.

Prompt 3: Module 15 - AI Personal Learning Engine and Adaptive Education System

You are an expert AI education system architect specializing in adaptive learning, machine learning personalization, education

Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation.

Completed modules include AR, translation, wearable compatibility, AI tutor, quiz, gamification, and 3D visualization.

Now implement Module 15: AI Personal Learning Engine and Adaptive Education System.

Goal: Create an intelligent learning system that analyzes user behavior, learning history, quiz performance, and interests to p

Requirements:

1. Create adaptive learning architecture under presentation/adaptive, data/adaptive, and domain/model.
2. Track objects scanned, categories explored, quiz scores, learning time, searches, difficult concepts, preferred language, and
3. Create LearningProfile with userId, totalObjectsLearned, averageQuizScore, favoriteCategories, weakAreas, learningLevel,
4. Analyze scans, quiz results, search history, and chatbot interactions to detect learning patterns.
5. Detect strong topics, weak topics, knowledge gaps, and learning preferences.
6. Generate AI recommendations, such as next plant anatomy topics or review material properties after low quiz scores.
7. Adapt quiz difficulty, explanation complexity, and learning suggestions for Beginner, Intermediate, and Advanced learners.
8. Create personalized learning roadmap: Current Level -> Recommended Topics -> Practice Activities -> Assessment -> Next L
9. Create dashboard showing progress, objects learned, strong areas, weak areas, and recommended next topic.
10. Integrate with AI Tutor for personalized responses.
11. Integrate with Quiz System for adaptive quiz generation and targeted practice.
12. Track daily learning time, weekly progress, object categories, quiz improvement, and learning consistency.
13. Add Room entities LearningProfileEntity and RecommendationEntity plus Firebase collections learningProfiles, recommen
14. Predict next suitable topic, learning difficulty, and improvement trend.
15. Protect user data and support user-controlled personalization settings.
16. Use background processing, coroutines, Flow, and cached analytics.
17. Test recommendations, profile updates, adaptive difficulty, dashboard accuracy, and AI personalization.

Do not modify existing AR, ML recognition, or visualization modules. Only add AI Personal Learning Engine capability.

Prompt 4: Module 16 - Advanced Computer Vision and Scene Understanding System

You are an expert Computer Vision AI engineer specializing in real-time object detection, scene understanding, mobile AI optimization, and AR development.

Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation.

Completed modules include AR, TFLite object recognition, AI Tutor, 3D visualization, translation, wearable compatibility, and AR overlay movement.

Now implement Module 16: Advanced Computer Vision and Scene Understanding System.

Goal: Upgrade the existing object recognition system into an intelligent vision system capable of detecting multiple objects, understanding scene context, and providing educational content.

Requirements:

1. Create advanced vision architecture under presentation/vision, data/vision, and domain/model.
2. Support multiple object detection, real-time tracking, classification, bounding boxes, and confidence scoring.
3. Integrate TensorFlow Lite Object Detection models, EfficientDet Lite, MobileNet SSD, and YOLO mobile models.
4. Keep inference mobile optimized, low latency, and battery efficient.
5. Track detected objects across frames, maintain object identity, reduce repeated recognition, and smooth AR overlay movement.
6. Analyze scenes such as kitchens and laboratories from detected objects.
7. Generate scene descriptions.
8. Analyze object relationships such as Plant + Pot + Soil.
9. Provide contextual educational content, recommended topics, safety information, and scientific explanations.
10. Support multiple information panels with colors/icons, distance scaling, and priority display.
11. Create a Vision UI with live camera, bounding boxes, labels, confidence scores, and scene information.
12. Optimize with frame skipping, quantization, GPU acceleration preparation, background inference, and memory optimization.
13. Integrate with ARCore, object detection, AR overlay, AI Tutor, Voice Explanation, and Learning History.
14. Support offline object detection, local inference, and cached knowledge retrieval.
15. Add DetectedSceneEntity with sceneId, userId, objectsDetected, timestamp, and locationContext.
16. Ensure camera data privacy and permission management.
17. Test multiple object detection, lighting, fast movement, low-end devices, and AR tracking stability.
18. Follow Clean Architecture, MVVM, Kotlin Coroutines, TensorFlow Lite best practices, and production-level mobile AI standards.

Do not replace existing object recognition system. Upgrade it into an advanced scene-aware computer vision platform.

Prompt 5: Module 17 - Cloud AI Infrastructure and Scalable Backend Architecture

You are an expert cloud architect specializing in scalable mobile backends, AI infrastructure, Firebase, cloud databases, machine learning, and cloud services.

Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation.

Completed modules include Android setup, MVVM, Compose, Firebase Auth, ARCore, TFLite, overlay, TTS, history, Room, Firestore, and analytics.

Now implement Module 17: Cloud AI Infrastructure and Scalable Backend Architecture.

Goal: Transform ROLA into a scalable cloud-powered educational platform capable of supporting large numbers of users, AI services, and analytics.

Requirements:

1. Create backend folders: authentication, user-management, object-knowledge, ai-services, analytics, recommendation, storage, and monitoring.
2. Design scalable architecture: Mobile Application -> API Gateway -> Backend Services -> Cloud Database -> AI Services.
3. Implement User, Object Knowledge, AI, and Analytics services.
4. Improve Firebase Authentication, Firestore, Storage, and Cloud Functions architecture.
5. Use database collections: users, objects, scanHistory, quizResults, learningProfiles, translations, 3DModels, and analytics.
6. Store object images, 3D models, AI model files, and educational resources with CDN, secure access, and version management.
7. Create AI Model Repository for TFLite models, versions, updates, and performance monitoring.
8. Prepare REST/GraphQL APIs for /users, /objects, /search, /recognition, /chat, /translate, /recommendations, and /analytics.
9. Build analytics pipeline for scans, learning time, quiz performance, search behavior, and AI interactions.
10. Add authentication tokens, role-based access control, API security, encryption, and database rules.
11. Prepare scalability for millions of users with Cloud Functions, serverless architecture, indexing, and caching.
12. Add monitoring preparation for Firebase Analytics, Crashlytics, and Performance Monitoring.
13. Create admin management for objects, educational content, 3D models, AI models, and analytics.
14. Implement offline + cloud synchronization: Local Database -> Sync Manager -> Cloud Backend -> Updated Data.
15. Test API performance, database scalability, security rules, cloud synchronization, and large user simulation.
16. Follow Clean Architecture, scalable backend principles, secure API design, and cloud best practices.

Do not modify existing mobile features. Only add scalable cloud infrastructure and backend capability.

Prompt 6: Module 18 - AI Knowledge Graph and Semantic Learning Network

You are an expert AI knowledge engineer specializing in knowledge graphs, semantic networks, ontology design, graph data

Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation.

Completed modules include Android setup, MVVM, Compose, Firebase Auth, ARCore, TFLite, AR overlay, TTS, history, Room, F

Now implement Module 18: AI Knowledge Graph and Semantic Learning Network.

Goal: Create an intelligent knowledge system that understands relationships between objects, scientific concepts, categories,

Requirements:

1. Create data/knowledgegraph with KnowledgeGraphRepository.kt, GraphDatabaseManager.kt, RelationshipEngine.kt, and .
2. Create domain/model with KnowledgeNode.kt, KnowledgeRelation.kt, ConceptGraph.kt, and LearningPath.kt.
3. Design KnowledgeNode fields: nodeId, name, type, description, category.
4. Support node types: Object, Material, Scientific Concept, Process, Application, Theory, Technology.
5. Model examples such as Copper -> Metal -> Electrical Conductivity -> Wiring -> Electronics.
6. Implement relationship types: is_a, made_of, used_for, related_to, part_of, causes, discovered_by, applied_in.
7. Build semantic search, e.g. query "How does electricity work?" resolves Electricity -> Conductors -> Copper -> Circuits.
8. Support Firebase-compatible local graph caching with collections knowledgeNodes, knowledgeRelations, and learningPath
9. Create graph visualization under presentation/knowledgegraph with KnowledgeMapScreen.kt, GraphNode.kt, and GraphC
10. Integrate AI Tutor with graph retrieval for grounded explanations such as why copper is used in wires.
11. Integrate Search System with semantic search in addition to exact object search.
12. Create AI Learning Path Generator for beginner and advanced routes.
13. Integrate with Adaptive Learning Engine to identify gaps, recommend next topics, and create learning sequences.
14. Add admin/AI-assisted knowledge expansion and scientific data update preparation.
15. Implement rule-based graph reasoning, including inferred relationships with exceptions preparation.
16. Optimize graph caching, indexed search, efficient relationship queries, and background loading.
17. Ensure verified educational content, admin-controlled updates, and privacy protection.
18. Test relationship accuracy, semantic search, graph navigation, AI Tutor responses, and recommendations.
19. Follow Clean Architecture, MVVM, graph database principles, Kotlin best practices, and scalable AI design.

Do not modify existing AR scanning or ML recognition modules. Only add AI Knowledge Graph and Semantic Learning Netwo

Prompt 7: Current Request - Numbered Prompts PDF

User request in Bangla: aikhan kar promptop gulo number soho akta pdf banai dao

Meaning: Create a PDF containing the prompts from here with numbering.